PROVISIONAL PATENT APPLICATION

SYSTEM AND METHOD FOR COMPUTING ON PRIVATE DATA WITHOUT EXPOSURE THROUGH SANDBOXED TRANSFORM EXECUTION AND ENCRYPTED RELAY ARCHITECTURE

Authors: Tom Tang

Organization: Edge Vector Foundation

Date: March 31, 2026

Applicant

Edge Vector Foundation
2328 Delmar Dr E, Seattle, WA 98102

Inventors

- Tom Tang, Seattle, WA

Field of the Invention

This invention relates to privacy-preserving computation on user-controlled data, and more particularly to a system and method for enabling third-party applications to derive computed results from private data without the data leaving the user's device or being exposed to any intermediary.

Cross-Reference to Related Applications

None (provisional filing).

Background of the Invention

Modern applications require access to user data — health records, financial information, personal documents, proprietary code — to provide useful functionality. Current architectures require users to upload data to centralized servers, where applications operate on it directly. This creates fundamental privacy problems: sensitive data is transmitted, stored, and processed on infrastructure the

user does not control. Once data reaches a platform, users have no meaningful control over how it is used, who sees it, or whether it is truly deleted.

Existing approaches to privacy-preserving computation, such as fully homomorphic encryption (FHE) and secure multi-party computation (SMPC), suffer from computational overhead that makes them impractical for general-purpose data operations. FHE imposes 1000x–10000x performance penalties. SMPC requires coordination between multiple parties and specialized protocols for each computation.

Self-hosted platforms (e.g., Nextcloud) shift custody from a cloud provider to a user-controlled server, but the server still has full access to plaintext data during processing. The server administrator can read, modify, or exfiltrate any file. There is no per-field access control, no schema-enforced policy, and no mechanism for third-party applications to compute on data without the server possessing it. Decentralized data pod systems (e.g., the Solid project) give users control over where data is stored, but applications that access the data receive the raw content — there is no transform layer, no per-field security labels, and no mechanism to return computed results without exposing the underlying data. Local-first databases using CRDTs enable offline-capable applications but do not address third-party access control: any application with read access sees all data. Encrypted database features (e.g., column-level encryption, always-encrypted queries) protect data from the database operator but require the querying application to hold decryption keys, defeating the purpose when the goal is to let applications compute without possessing the data.

There is a need for a practical system that enables third-party applications to compute on private data without the data leaving the user's control, using commodity hardware and standard programming models.

Summary of the Invention

The present invention provides a system and method for privacy-preserving computation comprising:

1. **A user-controlled database node** that stores data locally, encrypted at rest with keys derived from a user passkey, and serves as the sole authority over data access and computation.
2. **A schema-enforced access control layer ("folds")** that defines per-field permissions, trust distance requirements, security labels, and payment gates, evaluated at query time on the user's node.
3. **A sandboxed WebAssembly (WASM) transform execution engine** that enables third-party computation on private data within the user's node, where:
   - Transforms are content-addressed WASM modules registered in a public registry
   - Transforms execute in a sandboxed environment with no filesystem, network, or system access
   - Transforms operate on typed field values with declared input/output types verified at registration time
   - Transforms support composition: public standard library transforms and composed transforms chaining multiple operations

- The node executes transforms locally — plaintext data never leaves the node

4. **An encrypted relay layer** that routes end-to-end encrypted requests between third-party applications and user nodes, where the relay validates authentication tokens and enforces mechanical limits (access counts, expiration) but never sees plaintext data, never makes access control decisions, and never executes transforms.

5. **A reversible transform system** where:

   - Transforms optionally export an inverse function
   - Invertibility is verified at registration time via round-trip testing with type-appropriate test inputs
   - Verified-reversible transforms enable write-back through the transform to the source data
   - Non-reversible transforms store write values directly, marking the source link as stale

Detailed Description of the Invention

System Architecture

The system comprises three principal components: (1) a user-controlled database node running locally on the user's device, (2) a public transform registry that stores content-addressed WASM modules, and (3) an encrypted relay service that routes communications without accessing plaintext.

User Node

The user node is a local database that:

- Stores all user data encrypted at rest using AES-256-GCM with keys derived from a user passkey via Argon2id key derivation and HKDF per-purpose key derivation
- Maintains an Ed25519 keypair for node identity and end-to-end encryption with applications
- Evaluates all access control policies locally at query time
- Executes all WASM transforms locally in a sandboxed runtime
- Returns only authorized, optionally transformed results to requesting applications

The node is the sole point of decryption. No external system — including the relay — ever sees plaintext data.

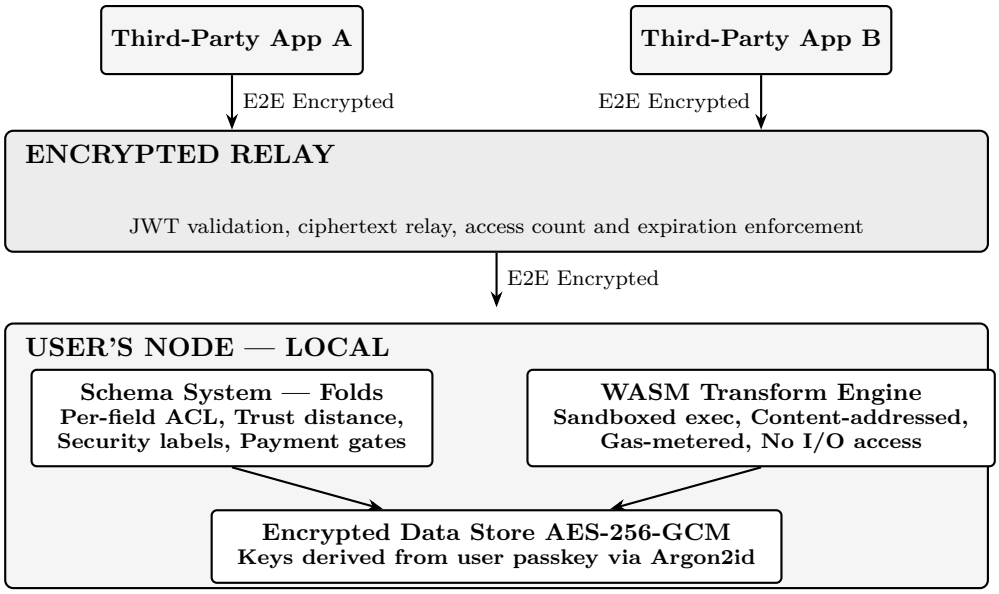


Figure 1: **System Architecture.** Third-party applications communicate with user nodes through an encrypted relay that validates tokens and relays ciphertext. All data decryption, access control, and transform execution occur on the user's local node.

Schema System (Folds)

Data is accessed through named, versioned interface definitions called "folds." Each fold specifies:

- Field names and types
- Per-field access control: read/write permissions, trust distance requirements, security label classifications, and optional payment gates
- Optional transform functions that compute derived values from source data

Applications do not access raw data. All access goes through folds that enforce field-level policies. The query engine evaluates access control before returning data; unauthorized fields are excluded from results such that the requester cannot distinguish "field has no data" from "field exists but access is denied."

Folds differ from traditional database views with row-level security in a critical respect: a fold enforces four conjunctive access control mechanisms simultaneously — trust distance, cryptographic capabilities, security labels, and payment gates — all evaluated at query time on the user's own node. Traditional views enforce a single permission model (e.g., role-based access) on a shared server. Folds compose into a directed acyclic graph where each node applies its own independent policy. Unauthorized fields produce no output — not an error, not a partial result, not an indication that the field exists — making the system indistinguishable from one where the field does not exist.

WASM Transform Engine

Transforms are WebAssembly modules that compute derived values from source data. The transform engine uses a WASM runtime with:

- Ahead-of-time compilation for near-native execution speed
- Fuel metering (gas limits) to prevent infinite loops
- No system interface — no filesystem, network, clock, or any system access
- Deterministic execution mode with NaN canonicalization
- Per-module memory limits

Data passing between host and guest occurs through serialized field values copied into WASM linear memory. The WASM module cannot access host memory or any data outside its sandbox.

Transform Composition Model

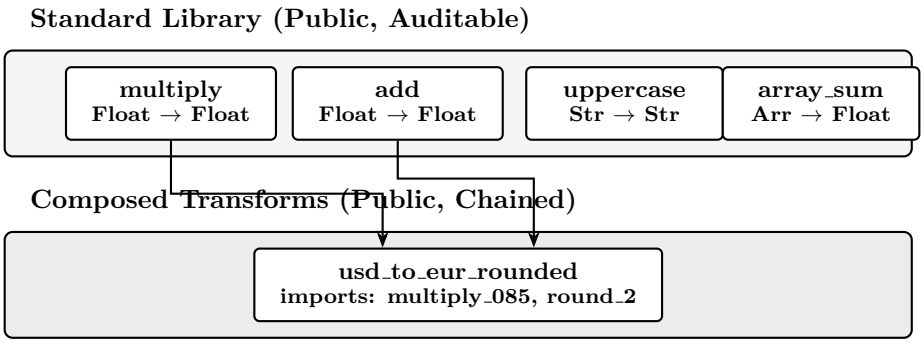


Figure 2: **Transform Composition Model.** Public standard library transforms are composed into chained transforms. All transforms are publicly auditable — anyone can inspect the source code, compile it, and verify the content hash matches the registered WASM module.

Standard Library (Public): Pre-compiled WASM modules for common operations (arithmetic, string operations, array aggregation, classification). Published with source code for full public verifiability. Anyone can compile the source and verify the content hash matches.

Composed Transforms (Public): WASM modules that import and chain standard library transforms. Composition is declared via WASM import tables, enabling verification of the entire computation chain without inspecting bytecode.

Because all computation occurs locally on the user's node — never on a remote server — there is no need for encrypted computation or key embedding. The WASM module operates on plaintext data within the node's local process, and results are only shared through the access-control layer (folds) which enforces per-field permissions before any data leaves the node.

Transform Verification

At registration time, the system performs:

1. **Type compatibility checking** — transform input type must match source field type; output type must match target field type
2. **Import resolution** — all imported transforms must exist in the registry
3. **Acyclicity verification** — the import graph must be a directed acyclic graph
4. **Gas limit validation** — max gas within system-configured bounds
5. **Reversibility verification** — if declared reversible, round-trip testing with type-appropriate synthetic inputs confirms invertibility

Content-Addressed Transform Registry

Each transform is identified by `SHA-256(wasm_bytes)`. This content-addressing provides:

- **Immutability:** once registered, a transform cannot be modified without changing its identity
- **Tamper evidence:** the node verifies `SHA-256(fetched_bytes) == expected_hash` before execution
- **Deduplication:** identical transforms produce identical hashes regardless of who registers them
- **Public auditability:** anyone can compile the published source and verify the hash matches

Encrypted Relay

The relay service (Exemem) performs strictly mechanical functions:

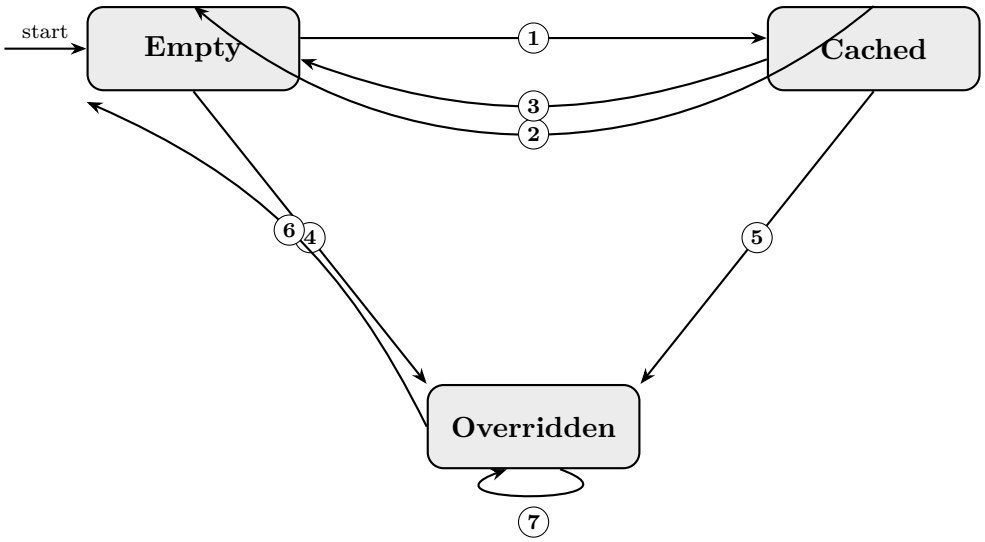
- Validates JWT authentication tokens (authenticity, expiration)
- Relays end-to-end encrypted ciphertext between applications and user nodes
- Enforces mechanical limits (access counts, time-based expiration)
- Stores encrypted blobs (node configuration, data backups)

The relay never decrypts data, never reads request or response bodies, never makes access control decisions, and never executes transforms. It is architecturally incapable of accessing plaintext.

Transform Field State Machine

Transform fields operate under a three-state machine:

- **Empty**: Never accessed. No value computed or stored.
- **Cached**: Value computed from source via WASM transform. Recomputes when source changes (invalidation via dependency map).
- **Overridden**: Value directly written by user. Source link is stale. Persists until explicitly cleared.



① **Read on Empty:** compute via WASM → Cached
② **Reversible write:** run inverse WASM, mutate source → Empty
③ **Source mutation:** invalidate cache → Empty
④ **Irreversible write** (from Empty): store directly → Overridden
⑤ **Irreversible write** (from Cached): store directly → Overridden
⑥ **Reversible write** (from Overridden): inverse WASM → Empty
⑦ **Source mutation** on Overridden: no change (stays Overridden)

Figure 3: **Transform Field State Machine.** Fields transition between Empty, Cached, and Overridden states based on read, write, and source mutation events.

Transitions:

- **Read on Empty:** compute via WASM, transition to Cached
- **Read on Cached:** return cached value
- **Read on Overridden:** return stored value
- **Reversible write:** run inverse WASM, mutate source, field resets to Empty
- **Irreversible write:** store directly, transition to Overridden
- **Source mutation:** Cached resets to Empty; Overridden unchanged

Data Flow — End-to-End Encrypted Computation

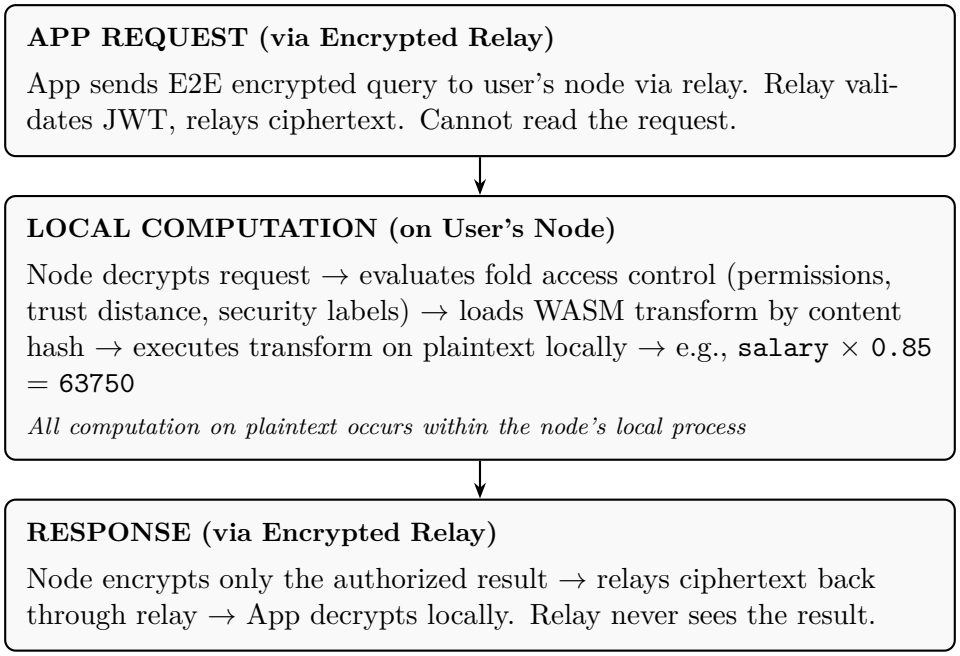


Figure 4: **End-to-End Data Flow.** An app sends an encrypted request through the relay to the user's node. The node decrypts, evaluates access control, executes the WASM transform locally on plaintext, and returns only the authorized result — encrypted — back through the relay.

Request Path:

1. Application sends E2E encrypted request through the relay to the user's node
2. Relay validates JWT and relays ciphertext — cannot read the request body

Local Computation:

1. Node decrypts the request
2. Node evaluates fold access control: permissions, trust distance, security labels, payment gates
3. Node loads the registered WASM transform by content hash and verifies `SHA-256(bytes) == expected_hash`
4. Node executes the transform on plaintext data within its local process
5. Node prepares the authorized result (only fields the requester is permitted to see)

Response Path:

1. Node encrypts the authorized result with the application's public key
2. Encrypted response is relayed back through the relay — relay cannot read it
3. Application decrypts locally

Claims

1. A computer-implemented method for computing on private data without exposure, comprising:
   a. defining schema-enforced access control interfaces (folds) over stored data, where each fold specifies per-field permissions using four conjunctive mechanisms: trust distance, cryptographic capabilities, security labels, and payment gates, all evaluated at query time;
   b. executing third-party computation as sandboxed WebAssembly modules locally on the user's node, wherein the modules have no access to filesystem, network, or system resources;
   c. returning only the authorized projection of computed results through the fold's access control layer, such that unauthorized fields are excluded in a manner indistinguishable from nonexistent fields.
2. The method of claim 1, wherein the WebAssembly transforms are content-addressed by the SHA-256 hash of their bytecode, registered in a public registry, and verified by the user's node before execution by recomputing the hash and comparing to the expected value.
3. The method of claim 1, wherein transforms support a composition model comprising:
   a. a public standard library of auditable transforms with published source code, where anyone can compile the source and verify the content hash matches the registered WASM module;
   b. composed transforms that chain standard library operations via declared WASM imports, where the import graph is verified as a directed acyclic graph at registration time;

   such that the entire computation chain is publicly verifiable without inspecting bytecode.
4. The method of claim 1, wherein all transform execution occurs locally on the user's node such that transforms operate on plaintext data within the node's local process, no data leaves the node during transform execution, and results are shared only through the access-control layer.
5. The method of claim 1, wherein data is stored on the user's node encrypted at rest with keys derived from a user passkey, and the node is the sole point of decryption.
6. The method of claim 1, wherein communications between applications and the user's node are routed through an encrypted relay that: stores only ciphertext and encrypted blobs; validates authentication tokens; is architecturally incapable of decrypting data or making access control decisions; and provides no degraded access mode when the user's node is offline.

7. The method of claim 1, further comprising a transform field state machine with three states — Empty, Cached, and Overridden — wherein:
   a. Empty fields are lazy-computed on first read via WASM transform execution;
   b. Cached fields are invalidated when source data changes via a dependency map;
   c. Overridden fields persist user-written values independently of source data changes;
   d. Reversible transforms verified at registration time via round-trip testing enable write-back through the inverse transform to mutate source data.
8. A system for privacy-preserving data computation comprising:
   a. a local database node configured to store data encrypted at rest and serve as the sole authority for decryption and access control;
   b. a schema system defining typed field-level access control interfaces;
   c. a sandboxed WASM runtime configured to execute content-addressed, publicly verifiable transform modules locally with no system access;
   d. a transform registry storing immutable WASM modules identified by the SHA-256 hash of their bytecode, enabling public auditability and tamper detection;
   e. an encrypted relay configured to route end-to-end encrypted communications between applications and user nodes without decrypting data.
9. The system of claim 8, wherein each fold field carries conjunctive access control comprising permissions, trust distance, security labels, and payment gates, all evaluated on the user's node at query time, with unauthorized fields excluded from results in a manner indistinguishable from nonexistent fields.

Abstract

A system and method for enabling third-party applications to compute on private user data without the data leaving the user's device or being exposed to any intermediary. The system comprises a user-controlled database node that stores data encrypted at rest and evaluates all access control policies locally; a sandboxed WebAssembly transform execution engine that runs third-party computation locally within the node using content-addressed, publicly auditable WASM modules with declared type signatures; and an encrypted relay that routes end-to-end encrypted communications between applications and user nodes without decrypting or interpreting data. Because all computation occurs on the user's local node rather than a remote server, transforms operate on plaintext within the node's process and require no encrypted computation. Results are shared only through policy-enforcing interfaces (folds) that evaluate per-field permissions at query time. The system enables practical privacy-preserving computation on commodity hardware without the performance penalties of homomorphic encryption or multi-party computation.

Prepared for provisional patent filing by Edge Vector Foundation.
Filing date: [TO BE DETERMINED]
This document constitutes a provisional patent application under 35 U.S.C. 111(b).